

NEXXUS FSM

Field Service Management Mobile App

Product Requirements Document — v1.0

React Native • iOS + Android • Integrated with NEXXUS POS Work Orders

Product	NEXXUS FSM	Version	1.0
Owner	MicroBooks Limited	Status	Draft — Ready for Development
Platform	React Native — iOS + Android	Date	9 August 2026
Backend	Supabase — shared with NEXXUS POS	Sync Mode	Real-time (Supabase Realtime)
Relationship	Separate app from NEXXUS POS Mobile	Shared Modules	Customers, Inventory, Work Orders, Staff

1. Overview

NEXXUS FSM is a dedicated Field Service Management mobile application built to extend the NEXXUS POS Work Order module into the field. It is a standalone app — separate from the NEXXUS POS mobile app — purpose-built for technicians working on customer sites.

Technicians receive job assignments from a dispatch manager, navigate to the customer location, track their time, log parts used against live inventory, capture before-and-after photos, collect the client's digital signature, collect cash payments, and record purchases made during the job. Every action syncs to NEXXUS POS in real time, giving the office complete visibility of field operations as they happen.

Why a separate app

NEXXUS POS Mobile is designed for counter-based retail — product grids, cart, checkout. Field technicians have a fundamentally different workflow: job queues, navigation, time tracking, photo capture, and on-site invoicing. Combining both into one app would compromise the experience for both user types. NEXXUS FSM shares the same backend, customers, inventory, and staff records — but presents a completely field-focused interface.

2. Goals & Non-Goals

Goals

- Give field technicians a purpose-built mobile app for managing assigned work orders end to end
- Enable dispatch managers to assign jobs and technicians to accept them with full accountability
- Track technician time on every job with clock in / clock out and pause reasons
- Capture GPS location at job start, arrival, completion and signature for verification
- Capture mandatory before-and-after photos on every job
- Collect the client's digital signature on job completion
- Log parts used directly against NEXXUS POS live inventory
- Allow technicians to collect cash payments from clients in the field
- Allow technicians to record material purchases made during a job
- Generate a complete job invoice covering labour, parts, travel and purchases
- Sync all field activity to NEXXUS POS Web in real time
- Work fully offline and sync automatically when connectivity is restored

Non-Goals

- NEXXUS FSM is not a replacement for NEXXUS POS Mobile — it does not handle retail counter sales
- It does not include a full product catalogue browse experience — only parts relevant to work orders
- It does not handle payroll, HR, or staff scheduling beyond job assignment
- It does not include route optimisation or multi-stop planning in v1.0
- It does not process card payments in v1.0 — cash collection only

3. User Roles & Permissions

NEXXUS FSM uses four distinct roles. Each role sees a different app experience. Roles are assigned in NEXXUS POS Web under Staff Accounts and sync to the FSM app.

Role	Can Do	Cannot Do
Admin / Dispatcher	Create and assign work orders; view all jobs across all technicians; reassign jobs; view all financials; approve purchases; access all reports; manage customers and inventory	—
Senior Technician	View and manage own assigned jobs; assign helpers to own jobs; collect cash; record purchases; log parts; complete jobs; view own job history and earnings	Cannot view other technicians' jobs; cannot create new work orders; cannot access company-wide reports
Technician	View and manage own assigned jobs only; collect cash; record purchases; log parts; complete jobs; view own job history	Cannot assign helpers; cannot view other technicians' jobs; cannot create work orders; cannot access reports
Helper / Apprentice	View jobs they are attached to; capture photos; log time; view job details and parts list	Cannot collect cash; cannot record purchases; cannot complete jobs; cannot see any pricing or financial information

Critical — Helper Role Financial Blackout

The Helper / Apprentice role must have ALL pricing, invoice totals, cash collection, and purchase functions completely hidden from the interface — not merely disabled. A helper should never see what a job is worth or what parts cost. This is enforced at the database level via Row Level Security and at the UI level by conditionally rendering financial components.

4. Work Order Lifecycle

Every job in NEXXUS FSM moves through a defined set of states. State transitions are logged with timestamp, user, and GPS coordinates.

#	Status	Triggered By	What Happens
1	DRAFT	Dispatcher (Web)	Work order created in NEXXUS POS Web. Customer, service description, scheduled date, and estimated parts added. Not yet visible to technicians.
2	ASSIGNED	Dispatcher	Dispatcher assigns to a technician. Push notification and email sent instantly. Job appears in technician's queue with an Accept / Decline action.
3	ACCEPTED	Technician	Technician taps Accept. Dispatcher notified. Job moves to active queue. If Declined, job returns to the pool with reason logged.
4	EN ROUTE	Technician	Technician taps Start Travel. GPS tracking begins. Travel time and distance recorded for invoicing.
5	ON SITE	Technician	Technician taps Arrive. GPS coordinates captured and stamped. Job clock starts. BEFORE photos required before work can proceed.
6	IN PROGRESS	Technician	Work underway. Technician logs parts used, records purchases, adds notes, and captures photos. Time accrues.
7	PENDING SIGN-OFF	Technician	Technician taps Complete Work. AFTER photos required. Invoice generated showing labour + parts + travel. Presented to client on screen.
8	SIGNED	Client	Client signs on the technician's device. Signature saved. Job clock stops. GPS captured. Receipt emailed automatically.
9	PAID / UNPAID	Technician	Technician records payment: Cash Collected (with amount), or Invoice to Account. Cash added to daily reconciliation.
10	COMPLETED	System	Job closed. Inventory deducted. Revenue recorded in NEXXUS POS. Appears in all reports. Dispatcher notified.

5. Application Screens

NEXXUS FSM has a bottom tab navigation with five primary sections. Screens adapt based on the logged-in user's role.

5.1 Screen Map

Tab	Screen	Purpose
Jobs	Job Queue	List of assigned jobs grouped by status: New (needs accept), Today, Upcoming, In Progress. Pull-to-refresh. Colour-coded priority badges.
	Job Detail	Full work order view — customer info, service description, scheduled time, address with Navigate button, parts list, photos, notes, action buttons based on status.
	Active Job	Live job screen while ON SITE / IN PROGRESS. Running timer, quick actions: Add Part, Record Purchase, Add Photo, Add Note, Complete Work.
Map	Job Map	Map view of today's jobs with pins. Tap pin to see job summary. Navigate button opens device maps app. Shows technician's current location.
Cash	Cash Drawer	Technician's running cash position: Opening float, Cash collected today, Purchases made today, Expected cash on hand. End-of-day reconciliation with variance.
	Record Purchase	Log a material purchase — supplier, amount, category, photo of receipt, link to job. Deducts from technician's cash on hand.
Inventory	Parts Lookup	Search NEXXUS POS live inventory. See stock levels. Add parts to the active job. Barcode scan supported.
	My Van Stock	Parts assigned to this technician's vehicle. Deducts automatically when used on a job. Request restock from Storeroom.
More	Profile	Technician details, role, today's stats — jobs completed, hours logged, cash collected.
	Job History	Completed jobs with date filters. Tap to view full job record including photos and signature.
	Timesheet	Hours logged per job per day. Weekly summary. Exportable.

5.2 Dispatcher Screens (Admin Role Only)

Screen	Function
Dispatch Board	All jobs across all technicians in a single view. Filter by status, technician, date, priority. Drag to reassign.

Create Work Order	Create a new job — select customer from NEXXUS POS customer list, add service description, set scheduled date and time, set priority, estimate parts, assign technician.
Team Map	Live map showing all technicians' current locations and their active job pins. Useful for assigning the nearest available technician.
Approvals	Queue of purchases made by technicians requiring approval above a configured threshold. Approve or reject with reason.

6. Core Feature Specifications

6.1 Job Assignment & Acceptance

- Dispatcher assigns a job in NEXXUS POS Web or in the FSM Dispatch Board
- Push notification fires instantly to the assigned technician's device
- Email notification sent simultaneously to the technician's registered email
- Notification includes: customer name, address, scheduled time, priority level, brief service description
- Technician opens the app and sees the job in the New section with Accept and Decline buttons
- Accept moves the job into the technician's active queue and notifies the dispatcher
- Decline requires a reason from a preset list (unavailable, wrong skillset, too far, other) and returns the job to the unassigned pool
- Jobs not accepted within a configurable window (default 30 minutes) trigger an escalation alert to the dispatcher

6.2 GPS Location Tracking

- GPS coordinates captured and stamped at four points: EN ROUTE start, ON SITE arrival, job completion, and client signature
- Continuous background location tracking active only between EN ROUTE and job completion — never outside working hours or between jobs
- Travel distance calculated from EN ROUTE start to ON SITE arrival — feeds the travel charge on the invoice
- Arrival verification — if the captured GPS is more than a configurable radius (default 200m) from the customer's registered address, flag the job for dispatcher review
- Technician must grant location permission on first launch — app explains clearly why it is required
- Location data is stored per job, not as a continuous trail, to respect technician privacy

6.3 Photo Capture

- BEFORE photos are mandatory — the job cannot move from ON SITE to IN PROGRESS without at least one photo
- AFTER photos are mandatory — the job cannot move to PENDING SIGN-OFF without at least one photo
- Additional photos can be added at any point during IN PROGRESS
- Each photo is timestamped and GPS-stamped automatically
- Photos compressed to a maximum of 1600px on the long edge before upload to conserve mobile data
- Photos upload to Supabase Storage in a job-specific folder with tenant-level RLS protection

- If offline, photos queue locally and upload automatically when connectivity returns
- Technician can add a short caption to any photo

6.4 Time Tracking

- Job clock starts automatically when the technician marks ON SITE
- Job clock stops automatically when the client signs
- Technician can manually pause the clock with a reason (break, waiting for parts, waiting for client)
- Paused time is recorded separately and excluded from billable hours
- Running timer visible at all times on the Active Job screen
- Total billable hours feed directly into the labour charge on the invoice
- All time entries roll up into the technician's weekly timesheet

6.5 Parts & Inventory

- Technician searches live NEXXUS POS inventory from within the job
- Barcode scanning supported using the device camera
- Adding a part to a job immediately reserves it against inventory
- On job completion, reserved parts are formally deducted from stock
- If a job is cancelled, reserved parts are automatically released back to available stock
- Van Stock — each technician can have parts assigned to their vehicle; using a van stock part deducts from the van, not the main storeroom
- Technician can request a van restock; the request appears in the Storeroom's transfer queue in NEXXUS POS
- Parts pricing is pulled live from the NEXXUS POS product catalogue — technicians cannot override pricing

6.6 Cash Collection

- Technician starts the day by confirming their opening cash float
- On job completion, the technician selects a payment method: Cash Collected, or Invoice to Account
- If Cash Collected, the technician enters the amount received and the app calculates change due
- Cash collected is added to the technician's running cash-on-hand balance
- A receipt is generated and emailed to the client automatically, and can be shared via WhatsApp
- At end of day, the technician performs a cash reconciliation: expected cash vs actual counted
- Any variance is flagged and requires a note from the technician
- Reconciliation submits to NEXXUS POS as a closed cash session for the technician

6.7 Field Purchases

- Technician can record a purchase made during a job — for parts bought from a local supplier
- Purchase form captures: supplier name, amount, category, job reference, and a mandatory photo of the receipt
- Purchase amount is deducted from the technician's cash-on-hand balance
- Purchases above a configurable threshold (default JMD \$10,000) require dispatcher approval before the job can be completed
- Approval request pushes to the dispatcher instantly; dispatcher approves or rejects with a reason

- Approved purchases are added to the job cost and can optionally be marked as billable to the client
- All purchases sync to NEXXUS POS for accounting and reconciliation

6.8 Digital Signature

- On job completion, the invoice summary is displayed on screen for the client to review
- Invoice shows: labour hours and rate, parts used with quantities and prices, travel charge, subtotal, tax, and total
- Client signs directly on the device screen using their finger or a stylus
- Signature captured as a PNG image and uploaded to Supabase Storage
- Client's printed name and the signature timestamp are recorded alongside the image
- Once signed, the job cannot be edited by the technician — only a dispatcher can reopen it
- A copy of the signed job sheet is emailed to the client automatically

6.9 Offline Mode

- The app must function fully offline — technicians frequently work in areas with poor connectivity
- Assigned jobs are cached locally when the app has connectivity
- All field actions (time tracking, photos, parts, notes, signature, cash) queue locally when offline
- A visible offline indicator shows the technician they are working offline and how many actions are pending sync
- When connectivity returns, queued actions sync automatically in chronological order
- Conflict resolution: server timestamp wins for job status; local data wins for technician-entered content
- Photos upload last, after all text data, to prioritise critical information

7. Job Invoice Structure

Every completed job generates a full invoice covering labour, parts, and travel. This is displayed to the client for signature and syncs to NEXXUS POS as a completed sale.

Line Item	Calculated From	Notes
Labour	Billable hours × technician hourly rate	Hourly rate set per technician or per service type in NEXXUS POS. Paused time excluded.
Parts	Sum of all parts logged against the job	Priced live from the NEXXUS POS product catalogue. Technician cannot override.
Travel	GPS distance × per-km rate, or flat call-out fee	Configurable per tenant — either distance-based or a flat call-out charge.
Field Purchases	Approved purchases marked as billable	Optional markup percentage configurable per tenant.
Subtotal	Sum of all above	—
Discount	Applied by technician if permitted by role	Requires dispatcher approval above a configurable percentage.

GCT / Tax	Subtotal × tenant tax rate	Uses the same tax rate configured in NEXXUS POS settings.
TOTAL	Subtotal – Discount + Tax	Amount presented to the client for signature and payment.

8. Database Schema Additions

NEXXUS FSM shares the same Supabase database as NEXXUS POS. The following tables are added. All are tenant-isolated via Row Level Security.

work_orders

```
create table work_orders (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
  ref               text not null,                -- WO-2026-0042
  customer_id       uuid references customers(id),
  assigned_to       uuid references staff(id),    -- technician
  assigned_by       uuid references staff(id),    -- dispatcher
  helper_ids        uuid[],                      -- attached helpers
  title             text not null,
  description        text,
  service_type       text,
  priority           text default 'normal',      -- low|normal|high|urgent
  status            text default 'draft',
  -- draft|assigned|accepted|en_route|on_site|in_progress
  -- |pending_signoff|signed|completed|cancelled
  scheduled_at      timestampz,
  address            text,
  address_lat        numeric(10,7),
  address_lng        numeric(10,7),
  accepted_at        timestampz,
  declined_reason    text,
  en_route_at        timestampz,
  arrived_at         timestampz,
  arrived_lat        numeric(10,7),
  arrived_lng        numeric(10,7),
  completed_at       timestampz,
  travel_distance_km numeric(10,2),
  billable_minutes   integer default 0,
  paused_minutes     integer default 0,
  labour_rate        numeric(12,2),
  labour_total       numeric(12,2),
  parts_total        numeric(12,2),
  travel_total       numeric(12,2),
  purchases_total    numeric(12,2),
  subtotal           numeric(12,2),
  discount           numeric(12,2) default 0,
  tax_amount         numeric(12,2),
  total             numeric(12,2),
  payment_method     text,                      -- cash|account
  amount_collected  numeric(12,2),
  signature_url      text,
  signed_by_name     text,
```



```
signed_at      timestampz,
created_at     timestampz default now(),
updated_at     timestampz default now()
);
```

work_order_parts

```
create table work_order_parts (
  id                uuid primary key default gen_random_uuid(),
  work_order_id     uuid not null references work_orders(id) on delete cascade,
  tenant_id         uuid not null references tenants(id),
  product_id        uuid references products(id),
  source            text default 'storeroom',      -- storeroom|van_stock
  name              text not null,                -- snapshot
  price             numeric(12,2) not null,       -- snapshot
  quantity          integer not null,
  line_total        numeric(12,2) not null,
  added_at          timestampz default now(),
  added_by          uuid references staff(id)
);
```

work_order_photos

```
create table work_order_photos (
  id                uuid primary key default gen_random_uuid(),
  work_order_id     uuid not null references work_orders(id) on delete cascade,
  tenant_id         uuid not null references tenants(id),
  photo_url         text not null,
  photo_type        text not null,                -- before|during|after|receipt
  caption           text,
  gps_lat           numeric(10,7),
  gps_lng           numeric(10,7),
  taken_at          timestampz default now(),
  taken_by          uuid references staff(id)
);
```

work_order_time_entries

```
create table work_order_time_entries (
  id                uuid primary key default gen_random_uuid(),
  work_order_id     uuid not null references work_orders(id) on delete cascade,
  tenant_id         uuid not null references tenants(id),
  staff_id          uuid references staff(id),
  started_at        timestampz not null,
  ended_at          timestampz,
  minutes           integer,
  entry_type        text default 'work',          -- work|break|waiting
  pause_reason      text,
  is_billable       boolean default true
);
```

field_purchases

```
create table field_purchases (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
  work_order_id     uuid references work_orders(id),
  staff_id          uuid references staff(id),
  supplier_name     text not null,
  amount            numeric(12,2) not null,
  category          text,
  description       text,
  receipt_url       text not null,                -- mandatory photo
  is_billable       boolean default true,
  markup_percent    numeric(5,2) default 0,
  approval_status   text default 'pending',        -- pending|approved|rejected
  approved_by       uuid references staff(id),
  approved_at       timestampz,
  rejection_reason  text,
  created_at        timestampz default now()
);
```

technician_cash_sessions

```
create table technician_cash_sessions (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
  staff_id          uuid references staff(id),
  opening_float     numeric(12,2) not null,
  cash_collected   numeric(12,2) default 0,
  purchases_total   numeric(12,2) default 0,
  expected_cash     numeric(12,2),
  actual_cash       numeric(12,2),
  variance          numeric(12,2),
  variance_note     text,
  opened_at         timestampz default now(),
  closed_at         timestampz,
  status            text default 'open'           -- open|closed
);
```

van_stock

```
create table van_stock (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
  staff_id          uuid references staff(id),    -- technician / vehicle
  product_id        uuid references products(id),
  quantity          integer default 0,
  min_threshold     integer default 2,
  updated_at        timestampz default now()
);
```

8.1 Row Level Security Policies

```
-- Technicians see only their own assigned work orders
create policy "Technicians see own jobs"
```

```

on work_orders for select
using (
  tenant_id = get_tenant_id()
  and (
    assigned_to = get_staff_id()
    or get_staff_id() = any(helper_ids)
    or get_staff_role() in ('admin','dispatcher')
  )
);

-- Helpers cannot see any financial columns – enforced via a view
create view work_orders_helper_view as
select id, ref, customer_id, title, description, service_type,
       priority, status, scheduled_at, address,
       address_lat, address_lng, arrived_at, completed_at
from work_orders;
-- No labour_total, parts_total, total, or pricing columns exposed

-- Only admins/dispatchers can approve purchases
create policy "Only dispatchers approve purchases"
on field_purchases for update
using (get_staff_role() in ('admin','dispatcher'));

```

9. Real-Time Sync Architecture

NEXXUS FSM and NEXXUS POS Web share the same Supabase project. Real-time synchronisation is achieved via Supabase Realtime WebSocket subscriptions on both sides.

Event	Origin	Real-Time Effect
Job assigned	Dispatcher — Web or FSM	Push notification + email to technician instantly. Job appears in queue without refresh.
Job accepted	Technician — FSM	Dispatch Board on Web updates status instantly. Dispatcher sees confirmation.
Status change	Technician — FSM	EN ROUTE / ON SITE / IN PROGRESS reflected live on the Web Dispatch Board.
Part added to job	Technician — FSM	Inventory reservation visible immediately in NEXXUS POS Web inventory screen.
Purchase submitted	Technician — FSM	If above threshold, push notification to dispatcher for approval.
Purchase approved	Dispatcher — Web	Technician's app updates instantly, unblocking job completion.
Job completed	Technician — FSM	Revenue appears in NEXXUS POS reports immediately. Inventory deducted. Dispatcher notified.
Cash reconciled	Technician — FSM	Cash session closes and appears in NEXXUS POS cash management.

Subscription Code Pattern

```
// FSM app subscribes to its own assigned jobs
const channel = supabase
  .channel(`tech-jobs-${staffId}`)
  .on('postgres_changes', {
    event: '*',
    schema: 'public',
    table: 'work_orders',
    filter: `assigned_to=eq.${staffId}`
  },
  (payload) => {
    if (payload.eventType === 'INSERT') {
      showNewJobNotification(payload.new);
    }
    refreshJobQueue(payload.new);
  })
  .subscribe();
```

10. Notifications

NEXXUS FSM uses push notifications for immediate field alerts and email for a permanent record. Both fire simultaneously on job assignment.

Trigger	Push	Email	Recipient
New job assigned	YES	YES	Assigned technician
Job accepted	YES	—	Dispatcher
Job declined	YES	YES	Dispatcher — with decline reason
Not accepted in 30 min	YES	YES	Dispatcher — escalation alert
Purchase needs approval	YES	YES	Dispatcher
Purchase approved/rejectedd	YES	—	Technician
Job completed	YES	YES	Dispatcher + signed job sheet to client
Arrival outside geofence	YES	—	Dispatcher — verification flag

11. Technical Stack

Layer	Technology	Notes
Framework	React Native (Expo)	Single codebase for iOS and Android. Expo managed workflow for faster builds and OTA

		updates.
Language	TypeScript	Type safety across shared data models with NEXXUS POS Web.
Backend	Supabase	Same project as NEXXUS POS — PostgreSQL, Auth, Realtime, Storage, Edge Functions.
Auth	Supabase Auth + PIN	Technician logs in once with email/password, then uses a quick PIN for daily access.
Offline storage	WatermelonDB or MMKV	Local cache of jobs and a queue for pending sync actions.
Location	expo-location	Foreground and background location. Background only active during EN ROUTE to completion.
Camera	expo-camera + expo-image-manipulator	Photo capture with compression to 1600px before upload.
Barcode	expo-barcode-scanner	Scan parts into a job from the device camera.
Signature	react-native-signature-canvas	Captures client signature as a PNG for upload to Supabase Storage.
Maps	react-native-maps	Job pins, technician location, deep-link to native maps for navigation.
Push	Expo Notifications	Unified push for iOS (APNs) and Android (FCM) through Expo's service.
Email	Supabase Edge Function + Resend	Same email infrastructure as NEXXUS POS receipts.
State	Zustand + TanStack Query	Lightweight state management with built-in caching and offline handling.

12. Development Phases

NEXXUS FSM is delivered in four phases. Each phase produces a working, testable app. Estimated total: 12 weeks with one experienced React Native developer.

PHASE 1

Foundation & Job Queue — 3 weeks

- Expo project setup, TypeScript configuration, navigation shell
- Supabase client integration and auth flow — email/password login plus PIN quick access
- Role detection and role-based navigation rendering
- Database schema deployed: work_orders, work_order_parts, work_order_time_entries
- RLS policies for all four roles including the helper financial blackout view
- Job Queue screen — New, Today, Upcoming, In Progress sections
- Job Detail screen — full work order view
- Accept / Decline flow with decline reasons
- Push notification setup via Expo Notifications

- Real-time subscription to assigned jobs

Phase 1 Milestone

A technician can log in, see their assigned jobs, accept or decline them, and receive push notifications. Dispatcher can assign jobs from NEXXUS POS Web and see acceptance status update live.

PHASE 2

Field Execution — 3 weeks

- Status transition flow — EN ROUTE, ON SITE, IN PROGRESS
- GPS capture at all four stamp points with geofence verification
- Background location tracking scoped to active job only
- Time tracking — clock start, pause with reason, resume, auto-stop on signature
- Running timer UI on the Active Job screen
- Photo capture — before, during, after with mandatory gates
- Photo compression and Supabase Storage upload
- Parts lookup with live inventory search and barcode scanning
- Add parts to job with inventory reservation
- Van Stock module and restock requests
- Map screen with job pins and navigation deep-link

Phase 2 Milestone

A technician can travel to a job, arrive, start work, log time, capture photos, and add parts — with everything syncing live to NEXXUS POS Web.

PHASE 3

Financials & Sign-Off — 3 weeks

- Job invoice generation — labour, parts, travel, purchases, tax
- Invoice review screen for client presentation
- Digital signature capture and upload
- Signed job sheet email to client via Edge Function
- Cash collection flow with change calculation
- Field purchase recording with mandatory receipt photo
- Purchase approval workflow with dispatcher push notification
- Technician cash session — opening float, running balance, end-of-day reconciliation
- Variance flagging and note requirement
- Inventory deduction on job completion
- Revenue posting to NEXXUS POS reports

Phase 3 Milestone

Full end-to-end job completion — technician completes work, client signs, cash is collected, inventory deducts, and revenue appears in NEXXUS POS reports in real time.

PHASE 4 Offline, Dispatch & Polish — 3 weeks

- Offline mode — local job cache and action queue
- Sync engine with conflict resolution and chronological replay
- Offline indicator with pending action count
- Dispatcher screens — Dispatch Board, Create Work Order, Team Map, Approvals queue
- Job History and Timesheet screens
- Technician profile and daily stats
- Escalation alerts for unaccepted jobs
- Performance optimisation and photo upload queue tuning
- End-to-end QA across all four roles
- App Store and Play Store submission builds

Phase 4 Milestone — SHIP

NEXXUS FSM is production-ready, works fully offline, supports all four roles, and is submitted to both app stores.

13. Acceptance Criteria

Test Case	Expected Result	Pass
Dispatcher assigns job to technician	Push + email received within 10 seconds	[]
Technician accepts job	Dispatch Board on Web updates status live	[]
Technician taps ON SITE	GPS captured, job clock starts, before-photo gate enforced	[]
Technician adds part from inventory	Stock reserved, visible in NEXXUS POS Web instantly	[]
Technician records purchase above threshold	Dispatcher receives approval push; job blocked until approved	[]
Client signs on device	Signature saved, clock stops, job sheet emailed to client	[]
Technician collects cash and completes job	Inventory deducted, revenue in NEXXUS POS reports instantly	[]
Helper logs in and opens a job	ZERO pricing, totals, or cash functions visible anywhere	[]
Technician works a full job in airplane mode	All actions queue locally; sync completes on reconnect	[]
Technician arrives 500m from registered address	Geofence flag raised; dispatcher	[]

	notified	
Technician closes cash session with variance	Variance flagged, note required, session syncs to NEXXUS POS	[]
Technician A attempts to view Technician B's job	RLS blocks access — job not returned by query	[]
Job not accepted within 30 minutes	Escalation push + email sent to dispatcher	[]
App runs 10-hour workday on a single charge	Battery drain acceptable; background GPS scoped correctly	[]

NEXXUS FSM Summary

4 phases • 12 weeks • React Native • iOS + Android • Real-time sync
GPS • Photos • Signatures • Time Tracking • Inventory • Cash • Purchases • Offline
Shared Supabase backend with NEXXUS POS — one database, two purpose-built apps

End of Document — NEXXUS FSM PRD v1.0 — MicroBooks Limited